MTA Transit Pulse

A Data Management Pipeline for NYC Subway Ridership Analysis

Juan Perez

Data Management Course · Stony Brook University

May 4, 2026

The Idea

What if I built a real data pipeline?

Collect publicly available data published by the Metropolitan Transportation Authority (MTA), store it in a relational database, and visualize it interactively.

What if I built a real data pipeline?

Collect publicly available data published by the Metropolitan Transportation Authority (MTA), store it in a relational database, and visualize it interactively.

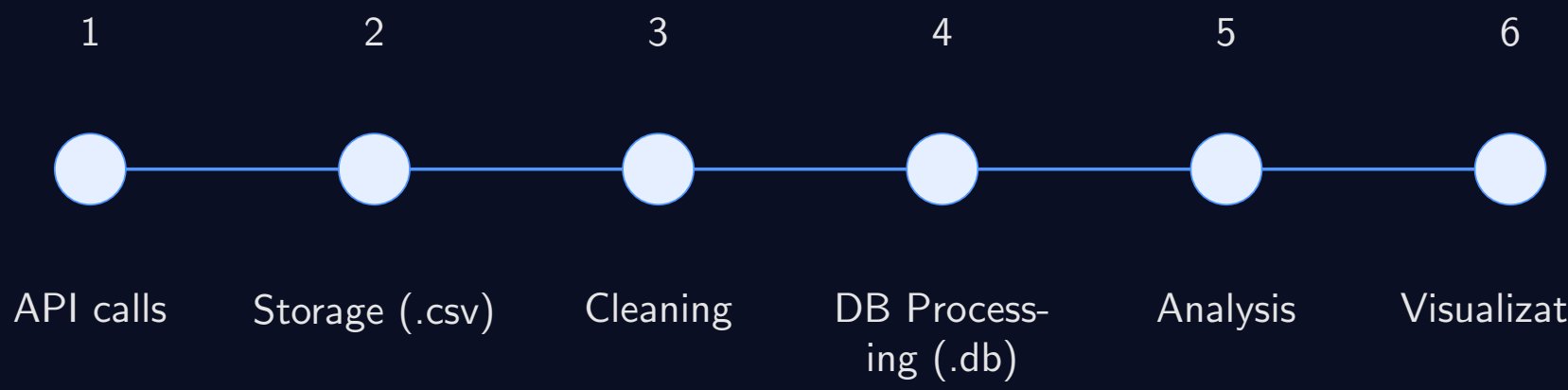
120M+
rows of data

428
unique stations

5 years
2020 – 2024

The Objective

Build an end-to-end data pipeline that pulls real MTA subway data, cleans and stores it in a local database, analyzes patterns and trends, and visualizes the results in a dashboard.



Cronological setup

The Dataset

MTA Subway Hourly Ridership: 2020–2024

data.ny.gov · dataset ID: wujg-7c2s

Column	Description
transit_timestamp	Hour in which the tap/swipe occurred (rounded down)
transit_mode	Subway, Staten Island Railway, or Roosevelt Island Tram
station_complex_id	Unique identifier for each station complex
borough	One of the five NYC boroughs
payment_method	OMNY or MetroCard
fare_class_category	Consolidated fare type (11 categories)
ridership	Total riders entering at this hour via this fare type
latitude / longitude	Station complex coordinates

Why This Dataset?

Strengths

- Real, production-grade data
- Pre-cleaned by the MTA (deduplicated, validated)
- Rich schema: time, location, fare type
- Spans COVID collapse & recovery (2020–2024)
- Free, public APIt

Challenges

- 120M rows (memory management required)
- API returns all values as **strings**
- Undocumented `georeference` column caused schema mismatch on first load
- Default API limit: 1,000 rows, token required

Stage 1 — Data Collection (collect.py)

Approach: SODA2 REST API with paginated requests

Query parameters sent on every request

```
params = {
        "$limit" : 500_000,    # rows per page
        "$offset": offset,     # advances each page
        "$where" : "transit_timestamp >= '2020-01-01T00:00:00'",
        "$order" : "transit_timestamp ASC"
}

headers = {"X-App-Token": TOKEN}   # loaded from .env
```

- **240 API requests** to collect all 5 years
- Each page saved as `raw_page_NNN.csv`
- **Resume capability**: skips pages already on disk
- 1s to 10s delay between requests to avoid throttling

Stage 2 — Cleaning (`clean.py`)

Design principle: raw data is not modified — clean data is a separate product written to `data/clean/`

Memory strategy: process one page (500K rows) at a time

- Peak RAM: ≈50MB regardless of total dataset size
- Output: one `clean_YEAR.csv` per year (≈5GB each)

Steps applied per page:

1. Cast `ridership` & `transfers` to numeric
2. Drop `georeference` (redundant with lat/lon columns)
3. Remove nulls in critical columns
4. Remove ridership < 0 or $> 100{,}000$
5. Strip whitespace from string columns
6. Derive `date`, `hour`, `day_of_week`
7. Deduplicate

Stage 3 — Database (db.py + load.py)

Schema (SQLite)

```
CREATE TABLE IF NOT EXISTS
        ridership (
        id INTEGER PRIMARY KEY,
        transit_timestamp TEXT,
        station_complex   TEXT,
        borough           TEXT,
        payment_method    TEXT,
        fareclass_category TEXT,
        ridership   INTEGER,
        date        TEXT,
        hour        INTEGER,
        day_of_week TEXT
        );
```

Loading strategy

- Chunks of 500K rows via `pd.read_csv(chunksize)`
- Loading process done per year
- WAL mode enabled for bulk write performance
- 3 indexes created: `date`, `station`, `hour`

Final result:

120,855,567 rows · 0 nulls in critical columns

Data Quality Verification

First queries run after loading to check on integrity

Row count by year

```
SELECT strftime('%Y', transit_timestamp) AS year,
COUNT(*) AS rows
FROM ridership
GROUP BY year ORDER BY year;
```

Year	Rows
2020	20,339,851
2021	23,281,980
2022	24,635,810
2023	25,585,413
2024	27,012,513
Total	**120,855,567**

Analysis — Six Questions

All queries executed in .ipynb via `pd.read_sql_query()`

Q1 Which 10 station complexes have the highest total ridership?

Q2 How does ridership vary by **hour of day**?

Q3 Which **day of the week** is busiest?

Q4 How does ridership compare across **boroughs**?

Q5 How has **OMNY** adoption grown vs MetroCard over time?

Q6 What does the **COVID recovery** look like year over year?

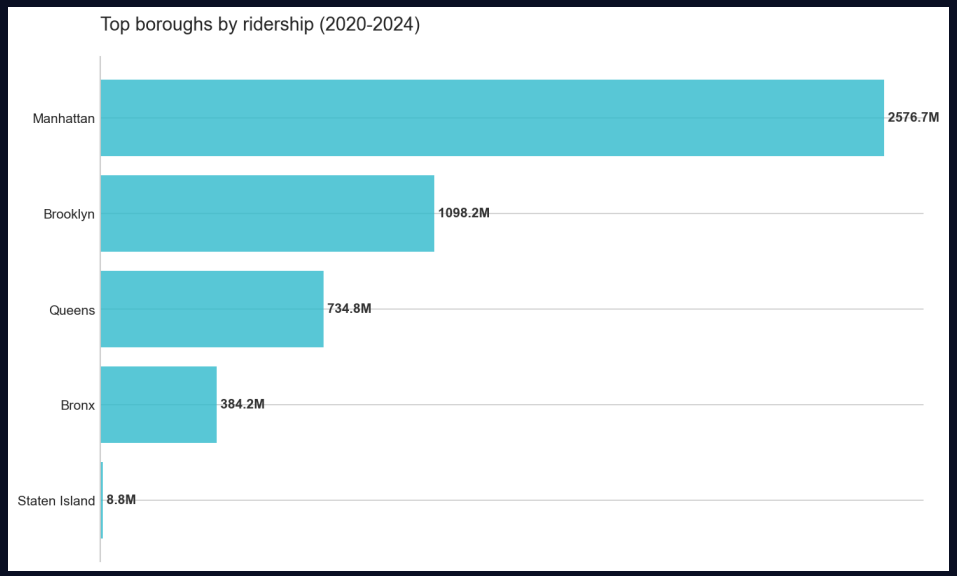
Q1 & Q4 — Stations and Boroughs

Top 5 stations by total ridership

Station	Total Riders
Times Sq–42 St	1.68B
Grand Central–42 St	1.15B
34 St–Herald Sq	974M
14 St–Union Sq	866M
Fulton St	707M

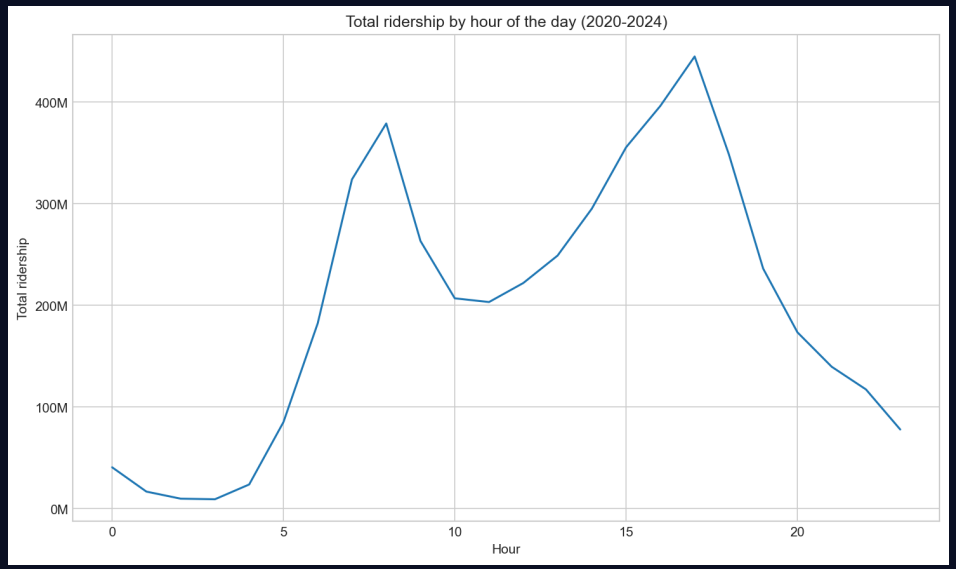
Penn Station appears **twice** under different line groupings (A/C/E and 1/2/3)

Borough comparison



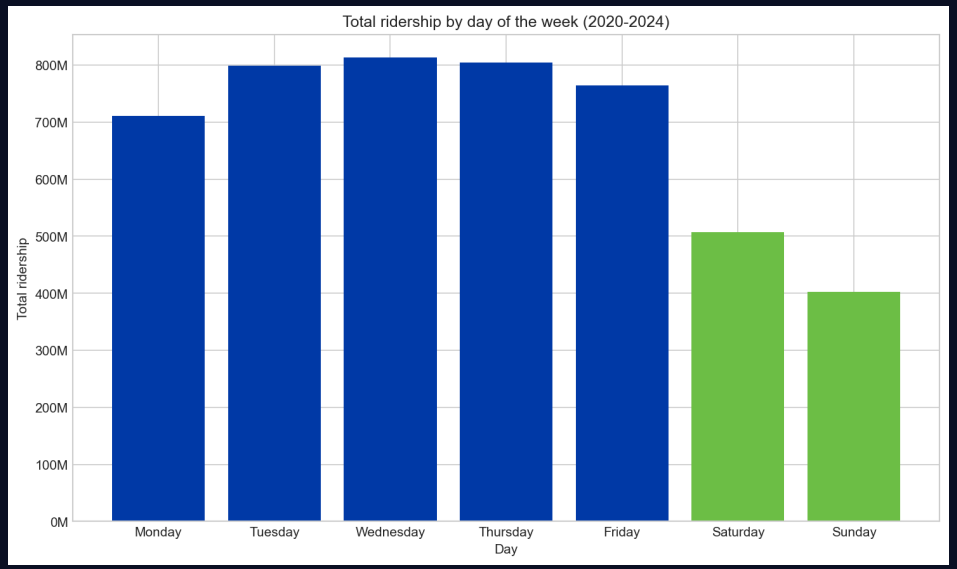
Q2 & Q3 — Temporal Patterns

Ridership by hour of day



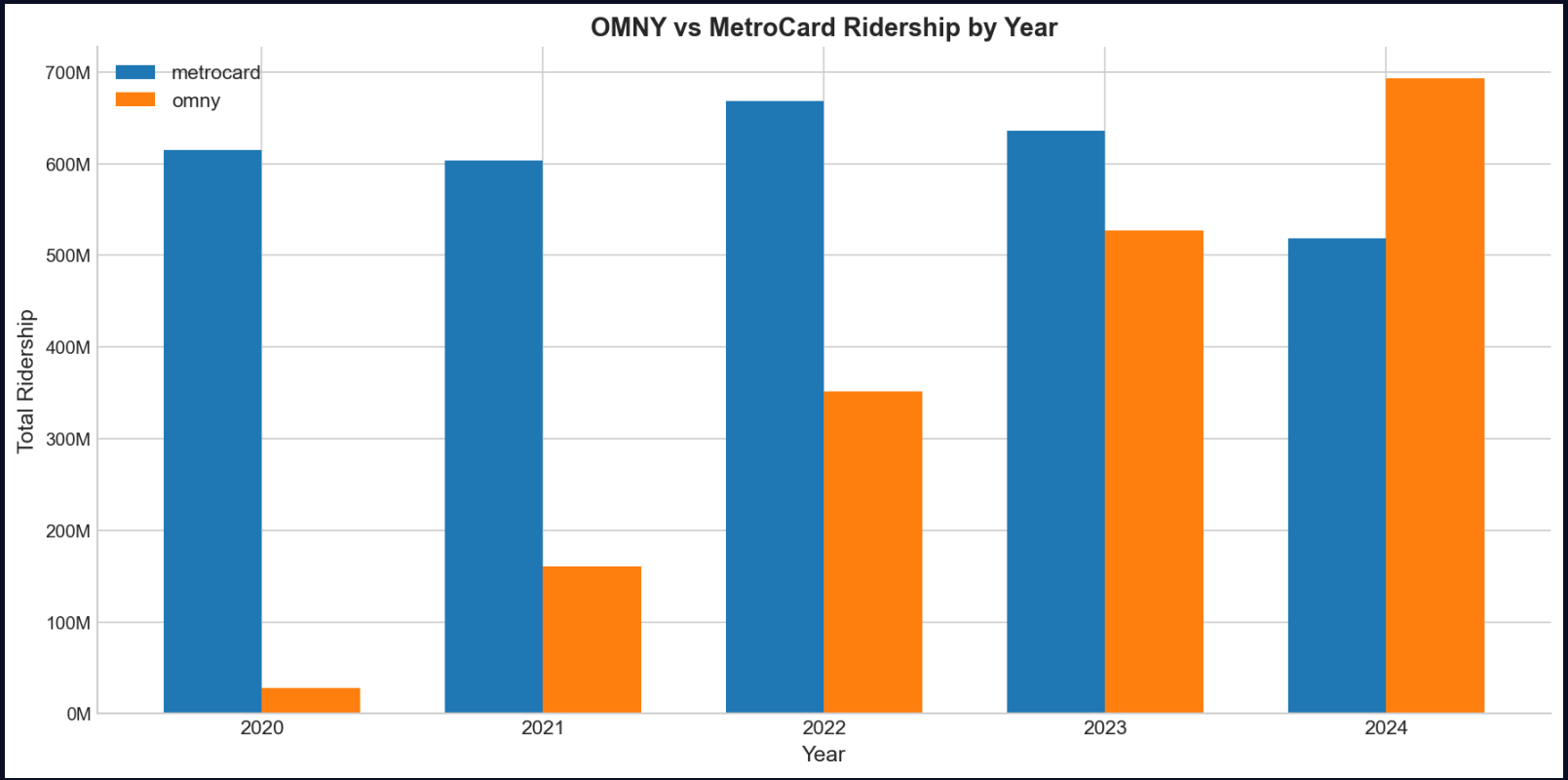
Classic double-peak: **08:00** morning rush, **17:00–18:00** evening rush

Ridership by day of week



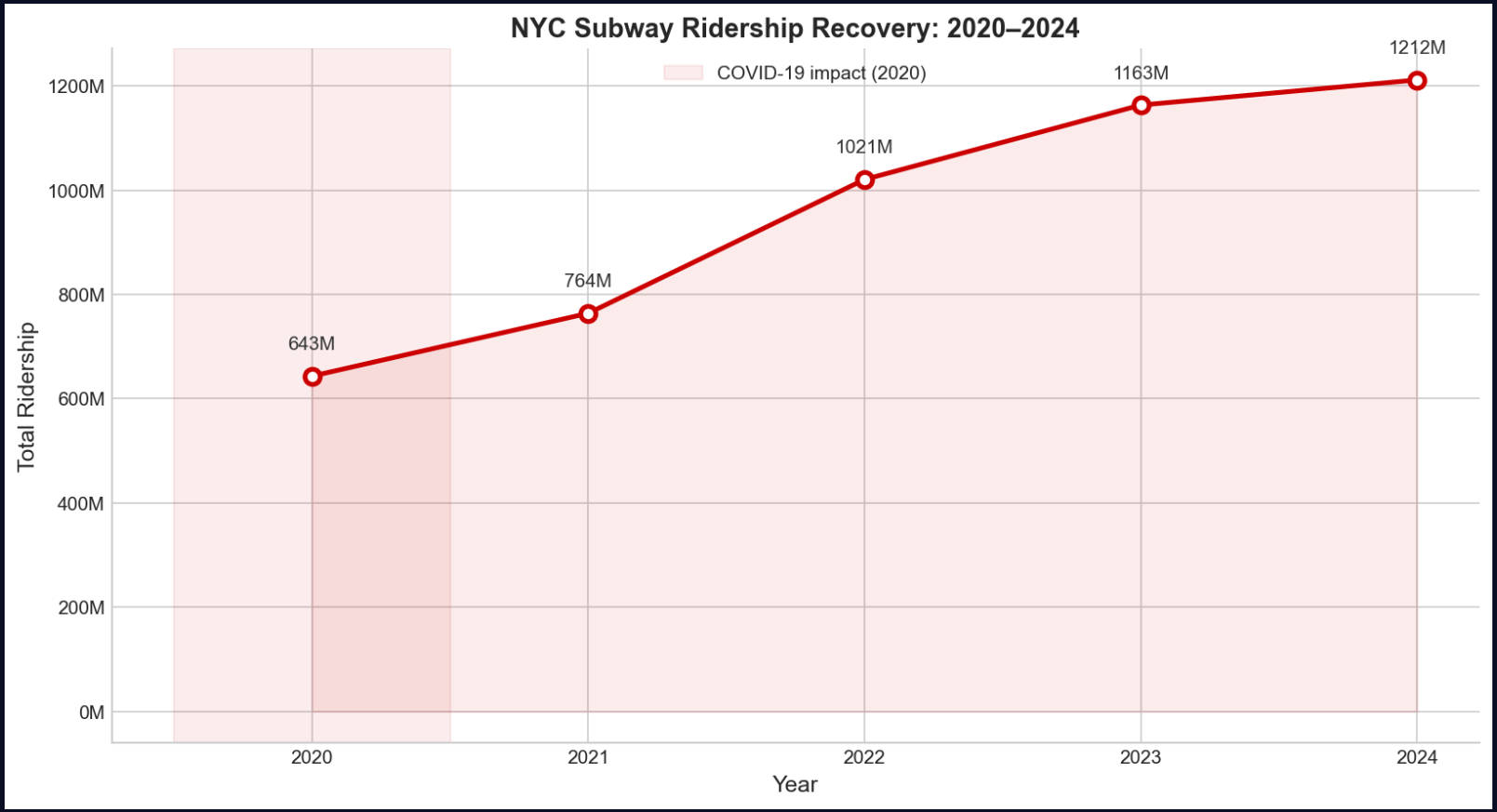
Friday is the busiest day — weekends show a clear drop vs weekdays

Q5 — OMNY vs MetroCard Adoption



OMNY launched in 2019. By 2024 it accounts for a **majority** of all subway rides — MetroCard adoption is in clear decline as the MTA phases out the legacy system.

Q6 — COVID-19 Ridership Recovery



Ridership collapsed in March 2020, reaching its lowest point during lockdowns. The dataset captures the full recovery arc — **2024 ridership is the highest in this dataset** at 27M rows, surpassing pre-pandemic baselines on many lines.

Streamlit Dashboard

Three interactive pages — all querying live from SQLite

Station Explorer

Pick any of 428 stations. See hourly & daily ridership profile. Filter by year.

System Trends

Network-wide patterns. Peak hours, busiest days, borough comparison, top 10 stations.

Recovery Timeline

Year-over-year recovery from COVID-19. OMNY adoption curve. Fare class breakdown.

Key design decisions

- `@st.cache_data` on all queries — avoids re-querying 120M rows on every user interaction
- Parameterized SQL queries — no string interpolation with user input

Engineering Challenges I

1. **Memory management**
   120M rows cannot fit in RAM simultaneously. Solution: page-by-page cleaning — never more than 50MB in memory at once regardless of total dataset size.

2. **Undocumented column**
   The API returned a `georeference` column absent from the official data dictionary. This caused a schema mismatch on first load. Fix: drop it during cleaning with `errors=‘‘ignore’’`.

3. **Type coercion**
   The SODA2 API returns every value as a string. Without explicit `pd.to_numeric()` casting, numeric comparisons and datetime operations silently fail.

Engineering Challenges II

4. **Transaction performance**
   Initial load tests showed slow inserts. Wrapping each year in a single transaction and enabling WAL mode reduced load time **5–10**×.

5. **API resume capability**
   A 240-request collection can crash halfway through. Solution: check for existing page files before each request and skip if already downloaded.

Key Findings

- **Times Square–42 St** is the busiest complex in the system by a wide margin — **1.68 billion riders** over 5 years
- The subway shows a clear **double-peak pattern** daily: 08:00 and 17:00–18:00, with a valley between 02:00–05:00
- **Manhattan** accounts for the overwhelming majority of system-wide ridership, followed by Brooklyn
- **OMNY** has overtaken MetroCard as the dominant payment method by 2024 — a complete payment ecosystem transition captured in the data
- Ridership grew **+32.8%** from 2020 to 2024, confirming near-complete post-COVID recovery visible directly in the numbers
- **Friday** is consistently the busiest day of the week — weekends carry noticeably fewer riders than weekdays

Tools used

Tech stack

- **Python 3.13** — pipeline scripts
- **pandas** — data cleaning & transformation
- **SQLite** — 23GB relational database
- **Streamlit** — interactive dashboard
- **matplotlib** — analysis charts
- **python-dotenv** — secure credential handling
- **Git + GitHub** — version control

Future Work & Closing

If this project were extended:

- Real-time GTFS delay feeds to correlate ridership drops with service disruptions
- NOAA weather enrichment — does rain increase ridership on certain lines?
- PostgreSQL migration for multi-user access
- Automated weekly ingestion via GitHub Actions — triggered every Monday when MTA publishes new data
- Station-level geospatial map using `latitude` / `longitude` columns

MTA Transit Pulse

120,855,567 rows
428 stations · 5 boroughs
2020 – 2024

github.com/jujopo/mta-transit-pulse

Thank you

Questions?